PUBLIC TECHNICAL PORTFOLIO

Car Rental CRM Platform

A private, pre-launch CRM and booking ecosystem for car rental companies

Field	Detail
Developer / Owner	Stefanos Ifoulis
Project status	Under active production development; public summary only
Document purpose	Resume and GitHub portfolio explanation without publishing source code
Prepared	July 06, 2026

Public boundary

This document describes product scope, architecture, engineering decisions, and implementation approach. It intentionally excludes proprietary source code, credentials, private infrastructure values, and business-sensitive implementation details.

Executive Summary

I built a full-stack car rental CRM platform designed for the operational reality of rental companies: reservations, fleet readiness, customer records, pricing, payments, service incidents, washing queues, and a public booking website all need to stay consistent while staff work quickly during busy rental periods.

The project is intentionally more than a simple CRUD dashboard. The backend uses a Spring Boot microservice architecture with service discovery, gateway routing, event synchronization, per-service persistence, and domain-specific scheduling logic. The CRM frontend is a React/TypeScript dashboard with role-aware navigation, multilingual support, live operational counts, booking workflows, and calendar scheduling. A separate public website and backend-for-frontend handle customer availability search, booking, payment, and synchronization back into the CRM.

Portfolio value

The strongest engineering work is in the end-to-end product design, the booking allocation algorithm and the service-owned data model design.

Repository Scope

The local project contains the current CRM frontend, a Spring Boot backend composed of multiple services, a public website frontend/backend pair, deployment resources, historical frontend work, data exports, images, SQL procedures, and migration documentation. Only this Public folder should be published while the product remains commercial and pre-launch.

Area	What it contains	Public-safe description
CRM frontend	Current React/Vite/TypeScript dashboard	Role-aware car rental operations interface for staff, managers, admins, and specialized users.
CRM backend	Spring Boot services for booking, availability, fleet, categories, prices, users, payments, employees, resources, discovery, and gateway routing	Service-oriented backend with separate domains, databases, and event-based synchronization.
Public website	Localized React website plus Spring Boot backend-for-frontend	Customer-facing availability search, booking wizard, payment flow, and CRM integration.
Resources and data	Images, Excel imports, SQL procedures, deployment files, and migration validation materials	Operational assets and runbooks used during development and pre-launch validation.

Product Capabilities

The CRM is organized around daily rental operations rather than abstract admin screens. The goal is to help staff answer practical questions quickly: which cars leave today, which cars return today, which bookings need confirmation, which cars need washing, what availability exists for a date range, and how an accident or service event affects existing reservations.

- Reservation creation and editing with availability search, customer details, drivers, price, flight, notes, and checklist support.
- Full calendar/scheduler views that group bookings by vehicle and expose status through operational colors.
- Daily operations dashboards for pickups, returns, washing, confirmations, and bookings that still need a vehicle.
- Fleet management for cars, categories, fuel/type metadata, parking locations, custom fields, file uploads, and images.
- Pricing and discount coupon management for category-based rental pricing and website-visible offers.
- Customer/user management with trust/block states, search, and synchronization to booking workflows.
- Service and emergency workflows for technical problems, incapacitated vehicles, accident handling, and replacement-car decisions.

- Payment tracking and external integrations for invoices, payment status, and public website checkout.

Architecture Overview

The architecture separates the CRM, the public website, and domain services so each part can evolve without turning the product into a single monolith. Service discovery and gateway routing make the backend navigable, while service-owned databases and events keep responsibilities clear.

Layer	Implementation	Why it matters
Frontend CRM	React 18, Vite, TypeScript, Ant Design Pro, React Router, React Query	Fast dashboard UI with typed feature slices, role-aware routing, caching, and operational screens.
API boundary	Spring Cloud Gateway with explicit routes and OAuth2/JWT resource-server security	Central authenticated entry point for CRM APIs while blocking internal service endpoints at the public gateway.
Service discovery	Netflix Eureka discovery server and clients	Services register dynamically and communicate by service name instead of hardcoded host routing.
Domain services	Spring Boot 3.4 / Java 21 microservices	Booking, availability, car, category, user, price, payment, employee, and resources domains stay independently owned.
Persistence	MySQL for transactional services; MongoDB for document-oriented fleet/user/category data	Database choice follows domain shape and historical operational needs.
Messaging	RabbitMQ fanout exchanges plus outbox event tables	Durable service synchronization and cross-domain projections without direct database coupling.
Public website	React/Vite website plus Spring Boot backend-for-frontend	Public booking, payment, localized content, and CRM synchronization are kept outside the internal CRM UI.
Deployment	Dockerfiles and Docker Compose orchestration	Local and single-host deployment planning with service containers and persistent volumes.

Conceptual flow

CRM staff UI -> API Gateway -> domain services -> service-owned databases. Public website -> website backend-for-frontend -> CRM services. Domain changes -> outbox records -> RabbitMQ -> subscribing services and read models.

Backend Implementation

The backend is implemented as a Maven parent project with Java 21 and Spring Boot 3.4.2. The service set includes discovery-server, gateway-server, availability-service, booking-service, car-service, category-service, employee-service, payment-service, price-service, resources-service, and user-service.

Service Boundaries

Service	Responsibility	Notable engineering work
booking-service	Reservations, statuses, calendars, daily reports, smart rearrangement, Excel imports, website booking intake	Contains the interval scheduling engine, booking workflows, availability mirror events, and status operations.
availability-service	Searchable availability/read model, category copies, price/sub-car/preparation mirrors	Answers customer and CRM availability questions using stored procedures/read models that must stay consistent with booking allocation.
car-service	Fleet records, parking, service windows, accident/service state, vehicle images/files	Mongo-backed fleet domain with natural license lookup and UUID surrogate identifiers.
category-service	Vehicle categories, types, fuels, image metadata, category filtering	Mongo-backed category ownership with events consumed by other domains.
price-service	Prices, category pricing, discounts/coupons, website-visible pricing data	Price ownership, coupon APIs, stored price calculation integration, event publication.
user-service	Customer records, extra fields, user trust/block state, website user flow	User identity data, telephone as unique natural field, and outbound user events.
payment-service	CRM payments, invoice-related integrations, payment cancellation/update flows	Payment ledger APIs and integrations with external accounting/invoicing paths.
resources-service	Image and file upload/download for cars and shared resources	Separates binary/resource handling from domain services.

Event and Outbox Pattern

Several services use an outbox table/collection to record domain changes and publish them asynchronously. Scheduled publishers read unprocessed events, send JSON messages through RabbitMQ exchanges, and mark events processed after successful delivery. This pattern reduces the chance of losing cross-service updates when a service writes its own database and then needs to notify other services.

- Category, car, user, booking, payment, price, and availability changes are propagated as domain events instead of direct cross-database writes.

- Website booking and payment events are staged locally and then sent to CRM services when ready.
- The architecture uses mirror/read-model tables where a service needs another domain's data for fast availability or booking decisions.

Booking Allocation Engine

A major custom part of the system is the **smart-rearrange engine** in the booking domain. It is modeled as a sweep-line interval scheduler: bookings, service windows, incapacitation windows, sub-rental windows, and already allocated bookings are converted into time-ordered events. The engine processes ending events before starting events at the same timestamp, freeing cars before assigning new ones.

- Allocates bookings to specific licenses while preventing overlapping reservations.
- Adds preparation and wash time so a returned car is not immediately treated as available.
- Adjusts pickup/return timing for operational transport distance between locations.
- Respects service, incapacitated, and sub-rental windows.
- Handles ready cars, upgrades, external cars, and unfit bookings as explicit outcomes.

> **Algorithm invariant**
> Availability search is a relaxation of the final booking allocation algorithm: if the system offers a car/category as available, the booking engine must be able to place the booking when it is created. This invariant is documented in the backend notes and protected by characterization tests around helper behavior.

Identifier and Data Consistency Work

The project moved toward a UUIDv7 primary-key standard so service identifiers are non-enumerable, cross-service references are not tied to another service's storage mechanism, and inserts remain roughly time-ordered. The current direction uses UUIDv7 identifiers generated in application code, binary UUID storage for MySQL-backed services, standard UUID representation for MongoDB, and human-friendly booking/payment reference numbers separately from technical IDs.

- Converted core service IDs toward UUIDv7 while keeping natural keys such as telephone and license plate as unique lookup fields rather than primary keys.
- Documented service-by-service rollout order because booking, availability, category, car, user, price, website, and payment references are coupled through events and mirrors.
- Validated stored-procedure rewrites for binary UUID behavior against legacy outputs to avoid silent overbooking or lost availability.

CRM Frontend Implementation

The current CRM frontend is a Vite/React/TypeScript application with a feature-sliced structure. It uses a single shared Axios API client, Keycloak token refresh, React Query for data fetching and cache invalidation, Ant Design/Pro Components for dense operations UI, and lazy-loaded routes for performance.

Frontend area	What I built	Implementation notes
Authentication and roles	Keycloak login-required provider, route guards, role-specific navigation	Roles include admin, manager, washer, and super-admin paths.
Operations shell	Side navigation, command palette, profile controls, app tour, language/theme controls	Designed around frequent operations first, rare admin actions under manage/hidden routes.
Dashboard cockpit	KPI cards, alerts, next pickups/returns, notes, live badge counts	Operational counts refresh periodically and reuse React Query cache keys.
Booking flow	Availability search step followed by customer/driver/details step	Availability search filters by dates, locations, category name/type/fuel/seats/transmission.
Calendar	Full-screen scheduler with filters, color legend, booking details modal, rearrange action	Uses Bryntum SchedulerPro and model-building tests for robust calendar data.
Fleet and users	Vehicle, parking, service, file/image, custom field, and customer management screens	CRUD operations are separated into typed API modules and query hooks.
Pricing/payments	Price management, discount coupons, booking payment modal, payment status checks	Payment data is fetched by booking and invalidated after mutations.
i18n and theme	English/Greek translations, dayjs locale switching, persisted dark/light mode	Non-Ant Design surfaces key off the same theme state.

Public Website and Backend-for-Frontend

The public website is a separate React/Vite application served under a website route namespace. It contains localized English/Greek marketing and informational pages, fleet category pages, search pages, a booking wizard, add-ons, customer information, Stripe payment flow, and booking confirmation paths.

The website backend is a Spring Boot backend-for-frontend that owns website-specific booking records, calls CRM services through WebClient clients, exposes public availability/booking/payment endpoints, and uses its own outbox publisher to synchronize website-created bookings, users, payments, and confirmation emails back into the CRM.

- Website availability search calls a website-facing availability endpoint with category and date filters.
- Website bookings are created locally first, then pushed to CRM booking services and linked back through CRM booking IDs.
- Stripe checkout sessions support advance or full payment paths, and successful payment can confirm the booking.
- Email confirmation is staged through the website outbox so customer communication is tied to booking state.

Testing and Validation

The project includes both implementation tests and validation artifacts. On the backend, the most important current tests are characterization tests for the smart-rearrange algorithm helpers and event-time calculations. On the frontend, Vitest tests cover route guards, typed API wrappers, calendar data mapping, cockpit summaries, checklist conversion, badge logic, and feature API calls.

Validation area	Coverage present	Purpose
Booking algorithm	Smart-rearrange helper and event tests	Protect interval scheduling behavior while refactoring complex allocation logic.
UUID standard	UUID generator tests and migration runbooks	Keep service ID generation consistent and migration decisions explicit.
Stored procedures	Binary UUID procedure validation harness and consolidated procedure script	Confirm feasibility/price procedure output remains identical after ID encoding changes.
Frontend API modules	Vitest tests for bookings, cars, users, categories, coupons, prices, payments, service, emergency, and today APIs	Catch endpoint shape changes and payload conversion issues.
Frontend behavior	Tests for role guards, calendar model building, cockpit summary, checklist state, and badge logic	Protect high-risk UI/domain transformation code.

Engineering Decisions

- Chose microservices because booking, availability, fleet, user, price, payment, and resources have separate ownership and scaling/consistency concerns.
- Used a gateway with explicit routes rather than discovery-generated public routes, so internal endpoints are not exposed through the public API boundary.
- Used service-owned data and event synchronization to avoid direct writes into another service's database.
- Kept the public website separate from the internal CRM so customer booking and marketing concerns do not pollute staff operations screens.
- Introduced UUIDv7 IDs to reduce enumerable-ID risk and prepare for safer public contracts across services.
- Added characterization tests before refactoring the booking allocation algorithm, because behavior changes there directly affect revenue and overbooking risk.

Current Status and Next Work

The product is under active pre-launch development. The system already has the main architectural shape and many core workflows, but it should not be represented publicly as a fully launched SaaS

product yet. The honest public framing is: private commercial CRM in production development, with working modules, validated architectural decisions, and remaining launch hardening.

Area	Status	Next work before launch
Core CRM workflows	Built across booking, fleet, customers, pricing, service, daily operations, and payments	Continue operational QA with real rental scenarios and refine edge-case UX.
Booking/availability consistency	Documented invariant and tested algorithm helpers	Add more DB-coupled and end-to-end golden-oracle tests.
Identifier migration	UUIDv7 direction implemented/documented across services with procedure validation	Complete coordinated full-stack reseed and any remaining website/front-end ID alignment.
Security	Keycloak gateway and role-aware frontend in place	Rotate/externalize any development credentials, harden public/CRM boundary, review least privilege.
Payments	CRM payment APIs and website Stripe flow in progress	Finalize webhook/idempotency handling, accounting integrations, and reporting plan.
Deployment	Docker and compose assets present	Finalize production environment variables, backups, monitoring, and domain/TLS setup.

Resume-Ready Summary

The following bullets are public-safe and can be adapted for a resume, portfolio page, or GitHub profile README.

- Designed and built a full-stack car rental CRM platform using React, TypeScript, Spring Boot, Java 21, MySQL, MongoDB, RabbitMQ, Keycloak, and Docker.
- Implemented domain services for reservations, availability, fleet, categories, pricing, customers, payments, employees, and resource/file management.
- Built a custom booking allocation engine that rearranges reservations across vehicle licenses while respecting preparation time, service windows, transport timing, sub-rentals, upgrades, and external-car fallback.
- Developed an operations-focused CRM frontend with role-based navigation, daily dashboards, booking wizard, full-screen scheduler, multilingual UI, dark/light theme, command palette, and tested API layers.
- Built a customer-facing booking website and backend-for-frontend with localized content, availability search, add-ons, customer details, Stripe checkout, email confirmation, and CRM synchronization.
- Led a UUIDv7 identifier migration strategy and validated stored-procedure behavior to protect booking/availability correctness during data-model changes.

Public GitHub Guidance

Because the source code is intended for a future paid product, the recommended public GitHub repository should contain only public-safe artifacts such as this document, a project README, selected redacted screenshots, architecture summaries, and non-sensitive progress notes. Do not publish application source code, environment files, service secrets, live database exports, customer data, internal logs, or deployment credentials.

Recommended public framing

Private commercial product. Public repository contains portfolio documentation only.